

HAW Hamburg – Faculty of Engineering and Computer Science

Lab Sensors – Sommersemester 2026

Lab Report:

GNSS as a Sensor with UART Interface
(Waveshare L76K GPS HAT)

Author(s):

Ahmed Mohamed

Matriculation No.: 2757098

Jamal Shamsan

Matriculation No.: 2756372

Htet Wai Yan Tun

Matriculation No.: 2757519

Supervisor:

Prof. Dr. Rasmus Rettig

Date of Experiment:

May 7, 2026

Group:

Group 3

Contents

1	Introduction	2
2	Part 1: Setup and Basic Operation	2
2.1	Task Summary	2
2.2	Technical Problem: Connecting 3.3 V and 5 V Systems via UART	2
2.3	Experimental Setup	3
2.4	Results	3
2.5	Hardware Block Diagram	4
2.6	Data Flow Diagram	5
2.7	Software Analysis	5
2.7.1	ArduWaveshare.ino – Arduino Firmware	5
2.7.2	simpleNMEA.py – Data Logging	6
2.8	NMEA Output – Answers to Questions	6
2.9	Map Confirmation	7
3	Part 2: Investigation of the UART Interface	7
3.1	Task Summary	7
3.2	Photo of the Experimental Setup	8
3.3	Oscilloscope Measurements	8
3.4	Answers to Questions	10
3.4.1	Why does communication still work at 3.3 V to 5 V?	10
3.4.2	Why would connecting TX (D11) damage the Waveshare module?	10
3.4.3	Time for one bit transferred	10
3.4.4	Annotation of the first 10 bytes	11
3.4.5	How often is a new position transferred?	11
4	Part 3: Understanding GNSS Strings	12
4.1	Task Summary	12
4.1.1	Structure of the Text File	12
4.1.2	Which lines repeat regularly?	13
4.1.3	Content of the NMEA Messages and Position Information	14
5	Part 4: Estimating GNSS Accuracy	15
5.1	Task Summary	15
5.2	Software Analysis: NMEAReaderAccuracy.py	15
5.3	Results	16
5.4	Calculation of Position Error in Metres	17
6	Conclusion	18
A	Source Code Listings	19
A.1	ArduWaveshare.ino	19
A.2	simpleNMEA.py	19
A.3	NMEAReaderAccuracy.py	19

1 Introduction

This laboratory experiment investigates the Global Navigation Satellite System (GNSS) as a sensor with a Universal Asynchronous Receiver Transmitter (UART) interface. The hardware used is the WaveShare L76K GPS HAT, which integrates the Quectel L76 GNSS module, connected to an Arduino Nano as a serial bridge to a laptop running Python for data acquisition and analysis.

The experiment is divided into four parts:

1. Setup and basic operation of the GNSS system.
2. Investigation of the UART interface with an oscilloscope.
3. Understanding and parsing NMEA GNSS strings.
4. Estimating GNSS position accuracy through statistical analysis.

2 Part 1: Setup and Basic Operation

2.1 Task Summary

The goal of Part 1 is to establish a working hardware and software connection between the WaveShare L76K GPS HAT and the laptop via an Arduino Nano, verify that valid NMEA sentences are received, and confirm that the decoded position corresponds to the actual physical location.

2.2 Technical Problem: Connecting 3.3 V and 5 V Systems via UART

Voltage Compatibility

The WaveShare L76K GPS HAT uses 3.3 V logic levels on its I/O pins, while the Arduino Nano GPIO lines operate at 5 V. Directly connecting a 5 V output to a 3.3 V input can permanently damage the lower-voltage device, because the input voltage exceeds the absolute maximum rating (typically $V_{CC} + 0.3 \text{ V}$) of the 3.3 V component.

For *reading* data from the GNSS module (GPS $\rightarrow$ Arduino): the GNSS TX line outputs at 3.3 V logic. A logic HIGH of 3.3 V is still interpreted as HIGH by the Arduino (whose $V_{IH,\min} \approx 2.1 \text{ V}$ for 5 V supply), so communication from GNSS to Arduino works safely without a level shifter.

For *writing* to the GNSS module (Arduino $\rightarrow$ GPS): the Arduino TX would output 5 V logic HIGH signals into the 3.3 V-rated input of the GNSS module. This exceeds the maximum input voltage and risks damaging the GNSS IC. Therefore, the TX line (D11) is intentionally left **unconnected** in this experiment.

2.3 Experimental Setup

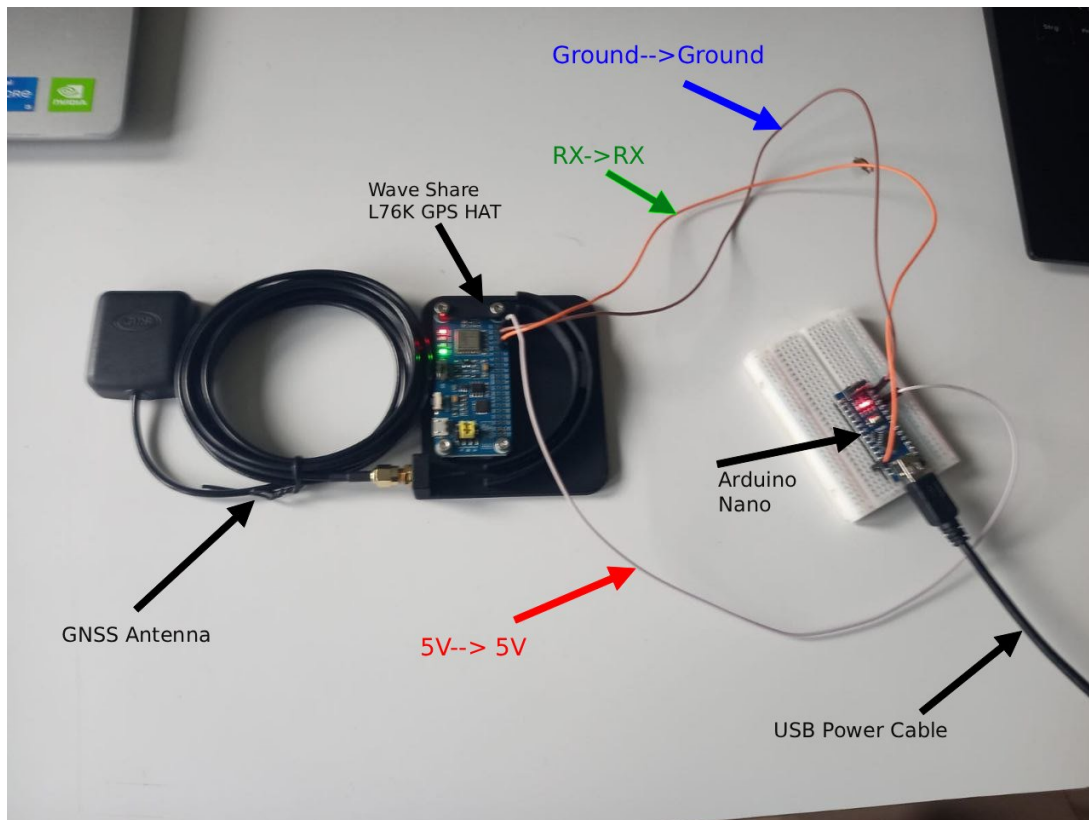


Figure 1: Hardware setup: Arduino Nano with WaveShare L76K GPS HAT and GNSS Antenna on breadboard.

2.4 Results

```
$GNRMC,064658.000,A,5333.4264,N,01001.4274,E,0.31,63.58,070526,,A*42
$GNVTG,63.58,T,M,0.31,N,0.57,K,A*1B
$GNGGA,064658.000,5333.4264,N,01001.4274,E,1,6,3.50,44.7,M,46.0,M,,*79
$GPGSA,A,3,05,21,11,09,,,,,,,,,3.64,3.50,0.99*0A
$BDGSA,A,3,06,11,,,,,,,,,3.64,3.50,0.99*12
$GPGSV,4,1,13,29,74,076,16,31,50,266,,28,45,222,,26,44,292,*71
$GPGSV,4,2,13,25,38,125,,18,35,177,,37,27,165,,05,18,088,33*76
$GPGSV,4,3,13,21,17,059,21,16,13,293,,04,12,324,,11,10,037,36*79
$GPGSV,4,4,13,09,05,354,19*4D
$BDGSV,2,1,06,26,70,235,,24,35,303,,11,23,054,39,06,22,067,35*61
$BDGSV,2,2,06,09,17,090,,02,03,106,27*6B
$GNLGL,5333.4264,N,01001.4274,E,064658.000,A,A*49
$GNRMC,064659.000,A,5333.4268,N,01001.4277,E,0.33,66.57,070526,,A*44
$GNVTG,66.57,T,M,0.33,N,0.61,K,A*16
$GNGGA,064659.000,5333.4268,N,01001.4277,E,1,6,3.50,44.7,M,46.0,M,,*77
$GPGSA,A,3,05,21,11,09,,,,,,,,,3.64,3.50,0.99*0A
$BDGSA,A,3,06,11,,,,,,,,,3.64,3.50,0.99*12
$GPGSV,4,1,13,29,74,076,16,31,50,266,,28,45,222,,26,44,292,*71
$GPGSV,4,2,13,25,38,125,20,18,35,177,,37,27,165,,05,18,088,32*75
$GPGSV,4,3,13,21,17,059,20,16,13,293,,04,12,324,,11,10,037,35*7B
$GPGSV,4,4,13,09,05,354,19*4D
$BDGSV,2,1,06,26,70,235,,24,35,303,,11,23,054,40,06,22,067,35*6F
$BDGSV,2,2,06,09,17,090,,02,03,106,26*6A
$GNLGL,5333.4268,N,01001.4277,E,064659.000,A,A*47
$GNRMC,064700.000,A,5333.4276,N,01001.4291,E,0.39,76.75,070526,,A*45
$GNVTG,76.75,T,M,0.39,N,0.72,K,A*1F
$GNGGA,064700.000,5333.4276,N,01001.4291,E,1,6,3.50,44.7,M,46.0,M,,*7D
$GPGSA,A,3,05,21,11,09,,,,,,,,,3.64,3.50,0.99*0A
```

Figure 2: screenshot of the recorded measurement data.

2.5 Hardware Block Diagram

Figure 3 shows the hardware structure including power supply and interfaces.

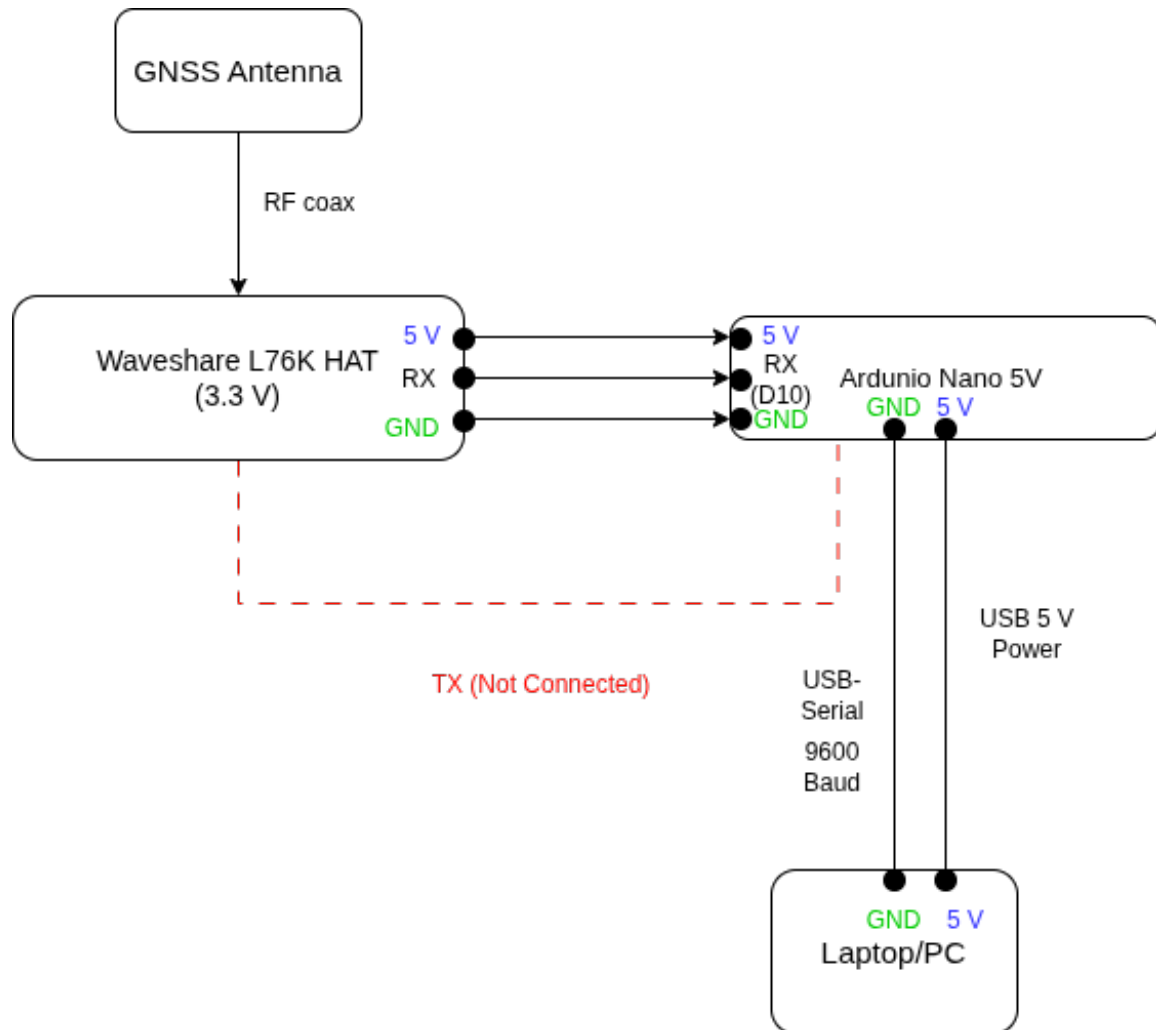


Figure 3: Hardware block diagram of the GNSS measurement system.

2.6 Data Flow Diagram

Figure 4 illustrates the data path from satellite signal to stored measurement, including how the timestamp is generated.

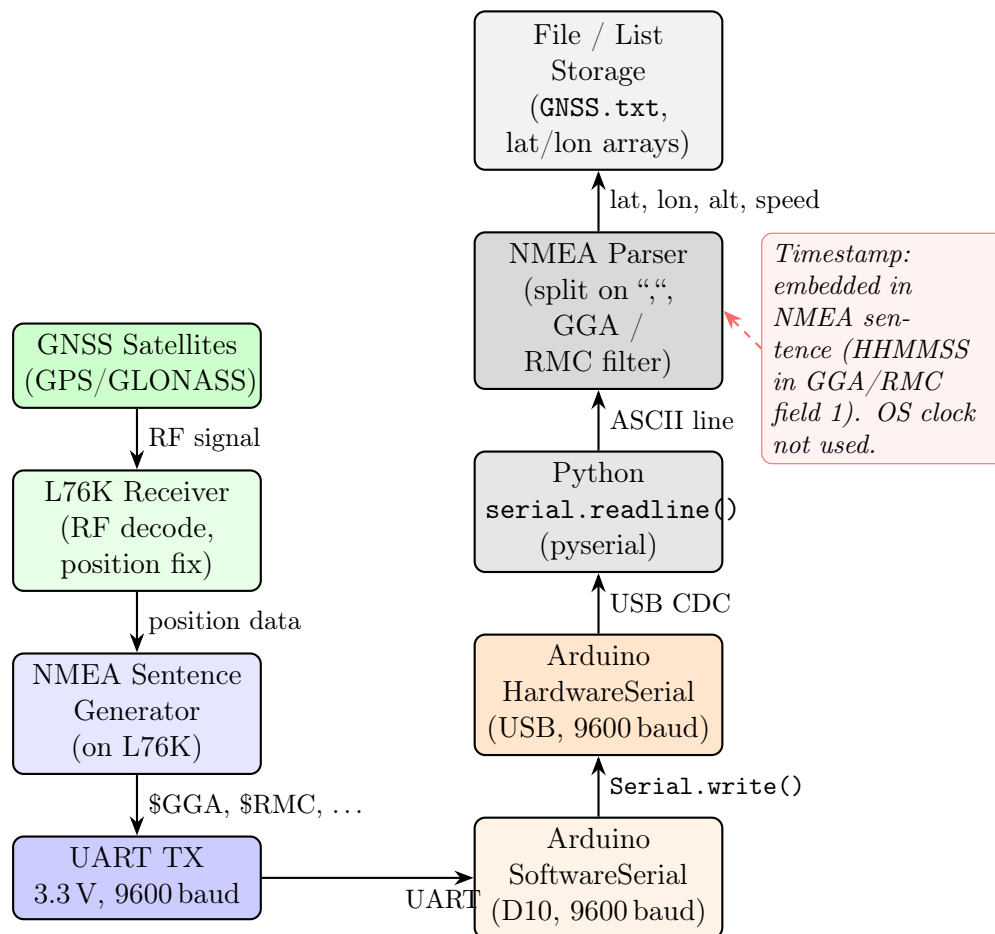


Figure 4: Data flow diagram from GNSS satellites to stored measurement data. The timestamp originates from the GNSS receiver’s internal clock (synchronised to satellite time), encoded in field 1 of GGA/RMC sentences.

2.7 Software Analysis

2.7.1 ArduWaveshare.ino – Arduino Firmware

```

1 #include <SoftwareSerial.h>
2
3 // Pin 10 = RX (receives from GNSS TX)
4 // Pin 11 = TX (NOT connected -- voltage safety)
5 SoftwareSerial mySerial(10, 11);
6
7 void setup() {
8     Serial.begin(9600);           // USB serial to laptop
9     mySerial.begin(9600);         // Software UART from GNSS module
10 }
11
12 void loop() {
13     if (mySerial.available()) {
14         Serial.write(mySerial.read()); // Forward byte-by-byte to USB
15     }
16 }

```

```

15     }
16 }

```

Listing 1: ArduWaveshare.ino – Arduino bridge firmware.

The Arduino acts purely as a **UART-to-USB bridge**. It reads raw bytes arriving on D10 from the GNSS module's TX line (using `SoftwareSerial` at 9600 baud) and immediately forwards each byte to the USB serial port (hardware UART, also 9600 baud) so the laptop can receive them. No processing or buffering is performed on the Arduino; the entire NMEA sentence reconstruction and parsing happens in Python.

2.7.2 simpleNMEA.py – Data Logging

```

1 import serial
2 ser = serial.Serial('/dev/cu.usbserial-1470', 9600)
3 with open('GNSS.txt', 'a') as file:
4     while True:
5         line = ser.readline().decode('ascii',
6                                     errors='replace').strip()
7         if line.startswith('$'):
8             print(line)
9             file.write(line + '\n')
10            file.flush()

```

Listing 2: simpleNMEA.py – NMEA sentence logger.

The script opens the serial port at 9600 baud and reads one line at a time. It filters for NMEA sentences (lines beginning with \$) and appends them to `GNSS.txt`. The `file.flush()` call ensures data is written to disk immediately in case of an unexpected program termination.

2.8 NMEA Output – Answers to Questions

How does a line output by the GNSS receiver start?

Every NMEA 0183 sentence starts with the dollar sign \$, followed by a two-letter talker ID (e.g. GP for GPS, GN for multi-constellation) and a three-letter sentence formatter. Example: `$GNGGA,103519.00,...*checksum`

Table 1: Common NMEA sentence descriptors found in the GNSS output.

Sentence	Full Name	Key Content
\$GxGGA	Global Positioning System Fix Data	Time, latitude, longitude
\$GxRMC	Recommended Minimum Specific GNSS Data	Time, date, lat, lon, speed (knots)
\$GxGSA	GNSS DOP and Active Satellites	Fix mode (2D/3D), satellite IDs
\$GxGSV	GNSS Satellites in View	Number of satellites, elevation, SNR
\$GxVTG	Course Over Ground and Ground Speed	Magnetic course, speed(knots and km/h)
\$GxGLL	Geographic Position – Latitude/Longitude	Lat, lon, time, status

2.9 Map Confirmation



3 Part 2: Investigation of the UART Interface

3.1 Task Summary

In Part 2, an oscilloscope probe is connected to the RX line (D10) of the Arduino. The UART signal from the GNSS module is captured at several timescales to characterise the transmission timing, telegram structure, and voltage levels.

3.2 Photo of the Experimental Setup

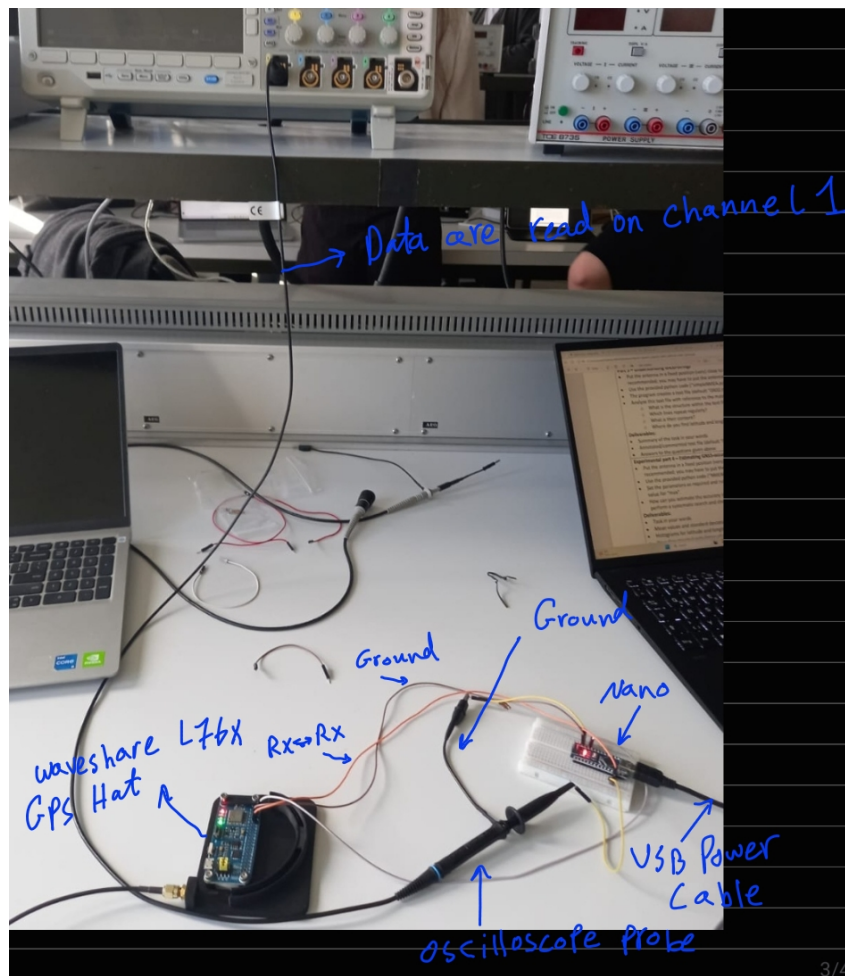


Figure 6: Oscilloscope capture of UART RX (D10) line

3.3 Oscilloscope Measurements

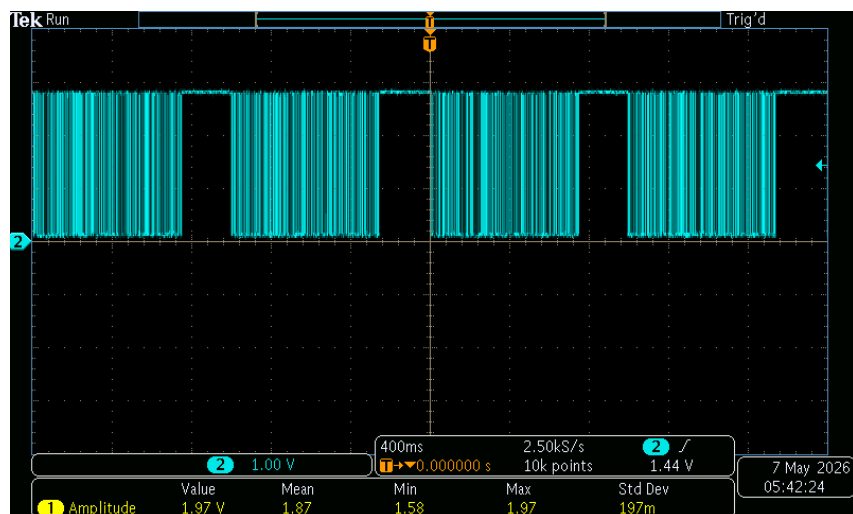


Figure 7: Oscilloscope capture of UART RX (D10) line at 400 ms/div – telegram repetition rate.

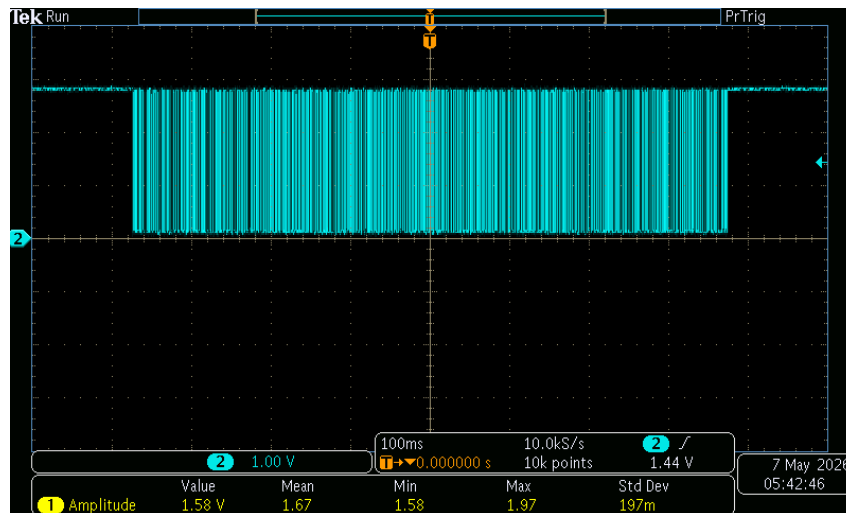


Figure 8: Oscilloscope capture of UART RX (D10) line at 100 ms/div – individual bytes visible.

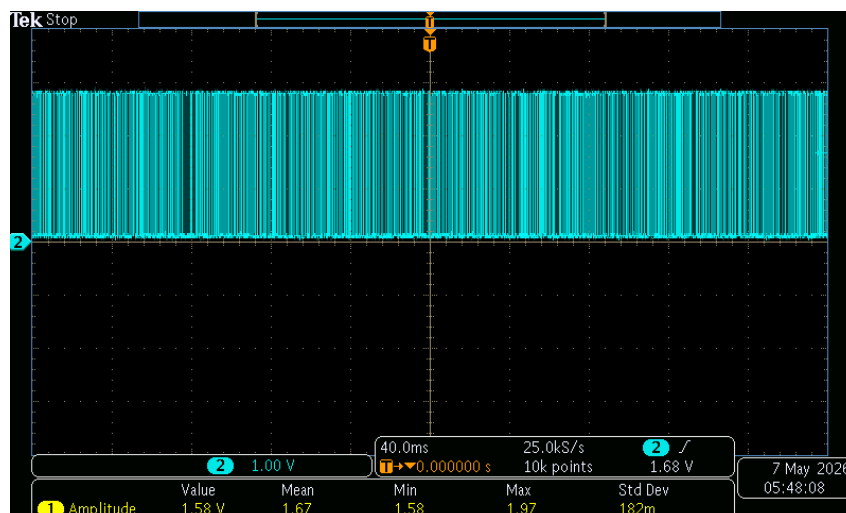


Figure 9: Oscilloscope capture of UART RX (D10) line at 40 ms/div – individual bytes visible.

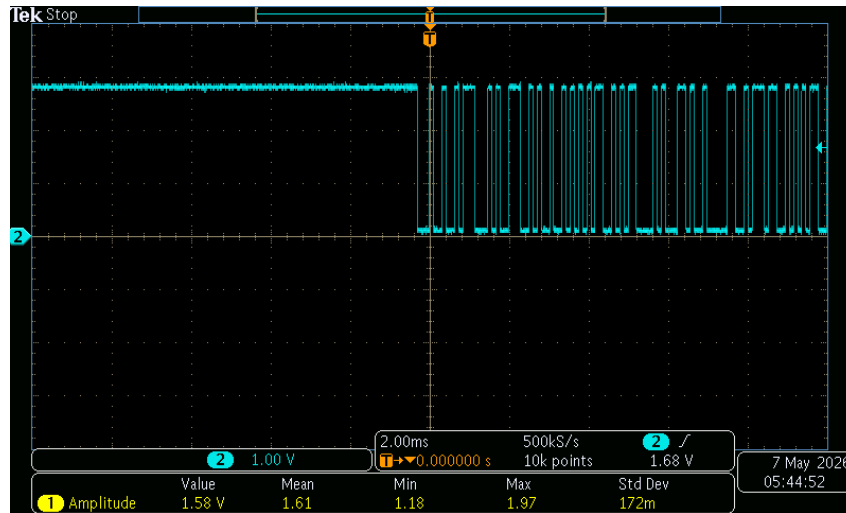


Figure 10: Oscilloscope capture of UART RX (D10) line at 2 ms/div – individual bytes visible.

3.4 Answers to Questions

3.4.1 Why does communication still work at 3.3 V to 5 V?

The Arduino's digital input threshold for logic HIGH (V_{IH}) with a 5 V supply is approximately:

$$V_{IH,\min} = 0.6 \times V_{CC} = 0.6 \times 5 \text{ V} = 3.0 \text{ V}$$

The GNSS module outputs a logic HIGH of 3.3 V, which exceeds this threshold. Therefore the Arduino correctly interprets the 3.3 V HIGH as a valid logic HIGH, and communication works without a level shifter.

3.4.2 Why would connecting TX (D11) damage the Waveshare module?

When D11 drives a logic HIGH it outputs 5 V. The L76K GNSS module has a maximum absolute input voltage of 3.6 V on its UART RX pin. Applying 5 V to this pin would violate the absolute maximum rating, potentially causing electrostatic-discharge-type damage to the CMOS input stage and permanently degrading or destroying the IC.

3.4.3 Time for one bit transferred

At 9600 baud the bit period is:

$$t_{\text{bit}} = \frac{1}{9600 \text{ baud}} \approx 104.2 \mu\text{s}$$

3.4.4 Annotation of the first 10 bytes

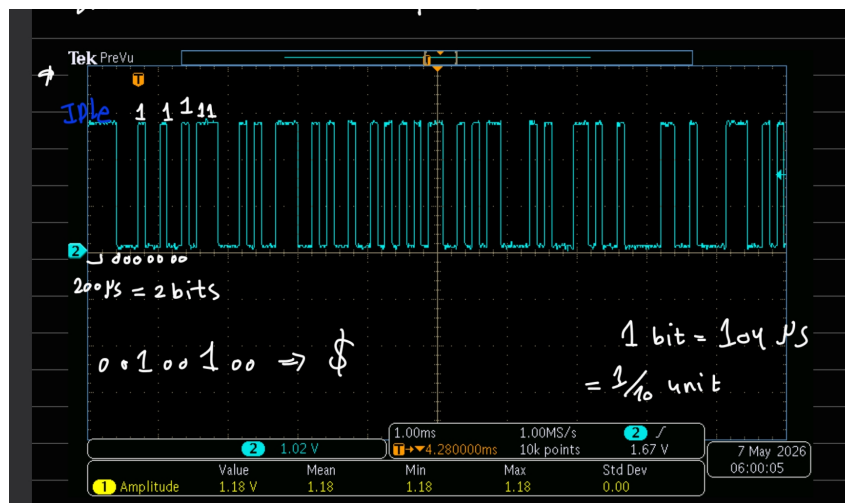


Figure 11: Oscilloscope capture of UART RX (D10) line at 1 ms/div – individual bytes visible.

UART uses 1 start bit (LOW), 8 data bits (LSB first), and 1 stop bit (HIGH) – total 1.042 ms per byte at 9600 baud. The first bytes transmitted after the line idles HIGH encode the NMEA talker ID and sentence type. For a \$GNGGA sentence the ASCII bytes are:

Table 2: First 10 bytes of a \$GNGGA NMEA telegram.

Byte #	ASCII	Binary (8-bit, LSB first)	Hex
1	\$	0010 0100	0x24
2	G	0100 0111	0x47
3	N	0100 1110	0x4E
4	G	0100 0111	0x47
5	G	0100 0111	0x47
6	A	0100 0001	0x41
7	,	0010 1100	0x2C
8	1	0011 0001	0x31
9	0	0011 0000	0x30
10	3	0011 0011	0x33

3.4.5 How often is a new position transferred?

The L76K module outputs NMEA sentences once per second (1 Hz) by default. This is visible on the 400 ms/div oscilloscope trace as groups of active data separated by approximately 1 s of idle (HIGH) line.

4 Part 3: Understanding GNSS Strings

4.1 Task Summary

In this part of the experiment, the GNSS antenna was placed in a fixed position close to the window in order to receive satellite signals. The Python program `simpleNMEA.py` was used to read the serial data from the Arduino/GNSS module via USB. The received NMEA messages were saved in the text file `GNSS.txt`.

The goal of this subtask was to understand the structure of the GNSS strings, identify the repeated NMEA message types, and find the latitude and longitude information inside the recorded file.

4.1.1 Structure of the Text File

The file `GNSS.txt` consists of many NMEA sentences. Each valid line starts with the character `$`. After this start character, there is a message identifier, for example `$GNRMC`, `$GNVTG`, or `$GNGGA`. The values inside the sentence are separated by commas. At the end of the line, there is a checksum after the `*` character.

Example of an NMEA Message

An example line from the file `GNSS.txt` is:

```
$GNGGA,064658.000,5333.4264,N,01001.4274,E,1,6,3.50,44.7,M,46.0,M,,*79
```

This line can be understood as follows:

Value	Meaning
\$GNGGA	message type
064658.000	UTC time
5333.4264	latitude
N	north
01001.4274	longitude
E	east
1	valid position fix
6	satellites used
3.50	HDOP
44.7 M	altitude in meters
*79	checksum

Table 3: Explanation of one GNGGA NMEA message

The general structure of an NMEA message is therefore:

```
$MessageType,value1,value2,value3,...*checksum
```

The NMEA reference manual confirms this general structure for messages such as `GGA`. According to the manual, the fields of a `GGA` message include the message ID, UTC time, latitude, longitude, position fix indicator, number of satellites used, HDOP, altitude, checksum, and the end of the message.

Note: This table is only for the GGA line. For the other lines, we use the other tables in the GNSS-documentation.

Table 1-3 GGA Data Format

Name	Example	Unit	Description
Message ID	\$GPGGA		GGA protocol header
UTC Time	002153.000		hhmmss.sss
Latitude	3342.6618		ddmm.mmmmm
N/S Indicator	N		N=north or S=south
Longitude	11751.3858		dddmm.mmmmm
E/W Indicator	W		E=east or W=west
Position Fix Indicator	1		See Table 1-4
Satellites Used	10		Range 0 to 12
HDOP	1.2		Horizontal Dilution of Precision
MSL Altitude	27.0	meters	
Units	M	meters	
Geoid Separation	-34.2	meters	Geoid-to-ellipsoid separation. Ellipsoid altitude = MSL Altitude + Geoid Separation.
Units	M	meters	
Age of Diff. Corr.		sec	Null fields when DGPS is not used
Diff. Ref. Station ID	0000		
Checksum	*5E		
<CR> <LF>			End of message termination

Figure 12: GGA Data Format

4.1.2 Which lines repeat regularly?

In the measured `GNSS.txt` file, the same group of NMEA messages repeats again and again. The repeated message types are:

```
$GNRMC
$GNVTG
$GNGGA
$GPGSA
$BDGSA
$GPGSV
$BDGSV
$GNGLL
```

The time value changes approximately every second:

```
064658.000
064659.000
064700.000
064701.000
064702.000
...
```

This shows that the GNSS receiver sends a new data block approximately once per second.

The `GSV` messages appear several times inside one block, for example:

```
$GPGSV,4,1,13,...
$GPGSV,4,2,13,...
```

```
$GPGSV,4,3,13,...
```

```
$GPGSV,4,4,13,...
```

This happens because satellite information is too long for one line, so it is split into several **GSV** lines. The manual also explains that multiple **GSV** messages may be required depending on the number of satellites tracked.

4.1.3 Content of the NMEA Messages and Position Information

The NMEA messages in the file have different functions.

The **RMC** message gives a short overview of the current GNSS data. It contains the time, valid status, position, speed, direction of movement, and date.

The **GGA** and **GLL** messages are especially important for the position, because they contain latitude and longitude. In the example line

```
$GNGGA,064658.000,5333.4264,N,01001.4274,E,...
```

the latitude is 5333.4264,N and the longitude is 01001.4274,E. The same position can also be found in the **RMC** and **GLL** lines.

The **GGA** message also gives extra information, for example whether the position is valid, how many satellites were used, the HDOP value, and the altitude. The **VTG** message gives movement information, mainly course and speed. The **GSA** messages show which satellites were used to calculate the position and contain DOP values. The **GSV** messages show the satellites visible to the receiver, including satellite number, elevation, azimuth, and signal strength.

Together, these repeated messages describe the current position, movement, and satellite situation of the GNSS receiver.

```

# RMC: Recommended minimum GNSS data.
# Contains time, valid status, position, speed, course and date.
$GNRMC,064658.000,A,5333.4264,N,01001.4274,E,0.31,63.58,070526,,A*42

# VTG: Course and speed over ground.
# Contains movement direction and speed.
$GNVTG,63.58,T,,M,0.31,N,0.57,K,A*1B

# GGA: Position and fix information.
# Contains time, latitude, longitude, fix quality, satellites used, HDOP and altitude.
$GNGGA,064658.000,5333.4264,N,01001.4274,E,1,6,3.50,44.7,M,46.0,M,,*79

# GSA: Satellites used for the position calculation and DOP values.
$GPGSA,A,3,05,21,11,09,,,,,,,,,3.64,3.50,0.99*0A

# GSA: Satellites used for the position calculation and DOP values.
# This line has the same GSA structure as described in the NMEA manual.
$BDGSA,A,3,06,11,,,,,,,,,3.64,3.50,0.99*12

# GSV: Satellites in view.
# The information is split into 4 GSV messages.
$GPGSV,4,1,13,29,74,076,16,31,50,266,,28,45,222,,26,44,292,*71
$GPGSV,4,2,13,25,38,125,,18,35,177,,37,27,165,,05,18,088,33*76
$GPGSV,4,3,13,21,17,059,21,16,13,293,,04,12,324,,11,10,037,36*79
$GPGSV,4,4,13,09,05,354,19*4D

# GSV: Satellites in view.
# The information is split into 2 GSV messages.
$BDGSV,2,1,06,26,70,235,,24,35,303,,11,23,054,39,06,22,067,35*61
$BDGSV,2,2,06,09,17,090,,02,03,106,27*6B

# GLL: Geographic position.
# Contains latitude, longitude, UTC time and valid status.
$GNGLL,5333.4264,N,01001.4274,E,064658.000,A,A*49

# Start of the next repeated NMEA block, one second later.
$GNRMC,064659.000,A,5333.4268,N,01001.4277,E,0.33,66.57,070526,,A*44
$GNVTG,66.57,T,,M,0.33,N,0.61,K,A*16
$GNGGA,064659.000,5333.4268,N,01001.4277,E,1,6,3.50,44.7,M,46.0,M,,*77
$GPGSA,A,3,05,21,11,09,,,,,,,,,3.64,3.50,0.99*0A

```

Figure 13: Annotated text file

5 Part 4: Estimating GNSS Accuracy

5.1 Task Summary

With the antenna stationary near a window, `NMEAReaderAccuracy.py` is run to collect a large number ($N = 5000$) of GGA fixes. The spread of latitude and longitude values around the mean is used to estimate the position accuracy of the L76K receiver.

5.2 Software Analysis: `NMEAReaderAccuracy.py`

```

1 # Convert NMEA DDDMM.MMMM to decimal degrees
2 degreeslat = int(parts[2][:2]) # integer degrees
3 minuteslat = float(parts[2][2:]) # decimal minutes
4 degreeslong = int(parts[4][:3])
5 minuteslong = float(parts[4][3:])
6
7 decimallat = degreeslat + minuteslat / 60

```

```

8 | decimallong = degreeslong + minuteslong / 60
9 |
10 | latitude.append(decimallat)
11 | longitude.append(decimallong)
12 |
13 | # After loop:
14 | latitude_mean = np.mean(latitude)
15 | longitude_mean = np.mean(longitude)
16 | # Standard deviations -> spread in decimal degrees

```

Listing 3: NMEAReaderAccuracy.py – statistical accuracy analysis (key section).

5.3 Results

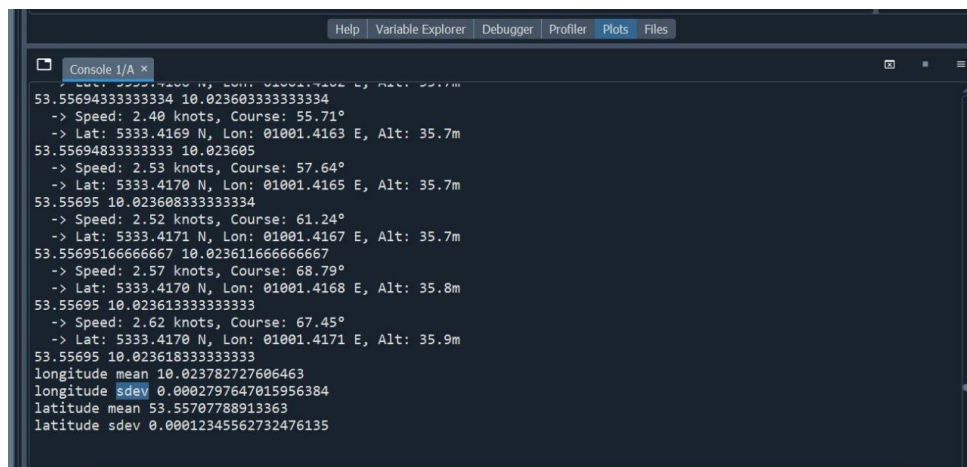


Figure 14: Screenshot of GNSS measurement results

Table 4: Statistical results of GNSS accuracy measurement ($N = 5000$ fixes).

Parameter	Latitude	Longitude
Mean (decimal degrees)	53.55707788913363° N	10.023782727606463° E
Standard deviation (decimal degrees)	0.00012345562732476135°	0.0002797647015956384°
Standard deviation (m)	13.74 mm	18.49 m

Histogram Placeholder

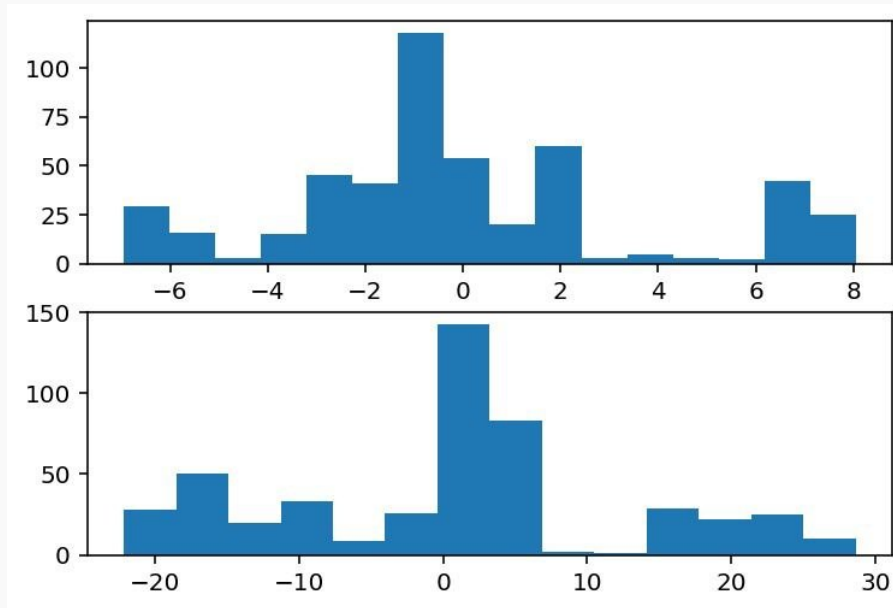


Figure 15: Histograms for latitude and longitude

- x-axis: Deviation from mean value / $\times 10^{-5}$ degrees
- y-axis: Number of measurements

5.4 Calculation of Position Error in Metres

The standard deviation in decimal degrees must be converted to metres. The relevant scale factors at a latitude of $\varphi \approx 53^\circ \text{N}$ (Hamburg) are:

- **Latitude:** $1 \approx 111,320 \text{ m}$ (nearly constant)

$$\sigma_{\text{lat,m}} = \sigma_{\text{lat,deg}} \times 111,320 \text{ m/deg}$$

- **Longitude:** $1 \approx 111,320 \times \cos(\varphi) \text{ m}$

$$\cos(53) \approx 0.6018 \implies 1 \approx 66,990 \text{ m}$$

$$\sigma_{\text{lon,m}} = \sigma_{\text{lon,deg}} \times 66,990 \text{ m/deg}$$

The combined 2D position accuracy (circular standard deviation) is:

$$\sigma_{2D} = \sqrt{\sigma_{\text{lat,m}}^2 + \sigma_{\text{lon,m}}^2}$$

This value represents the 1-sigma (68%) accuracy radius. The 95% confidence radius (common GPS specification metric) is approximately $2 \times \sigma_{2D}$.

$$\begin{aligned}\sigma_{\text{lat,m}} &= \underbrace{0.0001234556}_{\sigma_{\text{lat,deg}}} \times 111,320 \approx 13.74 \text{ m} \\ \sigma_{\text{lon,m}} &= \underbrace{0.0002797647}_{\sigma_{\text{lon,deg}}} \times 66,990 \approx 18.49 \text{ m} \\ \sigma_{2D} &= \sqrt{(13.74)^2 + (18.49)^2} \approx \boxed{23.03 \text{ m}}\end{aligned}$$

The measured 2D position spread of approximately 23 *m* is significantly larger than the typical open-sky accuracy specification of the L76K GNSS receiver ($\leq 2.5 \text{ m}$). The increased positioning uncertainty is mainly caused by the indoor near-window measurement setup, limited satellite visibility, and multipath reflections from surrounding structures.

6 Conclusion

This experiment demonstrated the complete signal chain from GNSS satellite signals to a statistical accuracy analysis on a laptop:

- The Waveshare L76K GPS HAT successfully outputs standard NMEA 0183 sentences at 1 Hz via its UART TX line at 3.3 V logic.
- The Arduino Nano serves as a safe, transparent UART-to-USB bridge. Only the RX direction (GPS→Arduino) is connected to avoid 5 V damage to the 3.3 V GNSS module.
- Oscilloscope measurements confirmed the 9600 baud timing ($\approx 104 \mu\text{s/bit}$) and the 1 Hz repetition rate of NMEA telegram groups.
- NMEA sentence parsing (GGA, RMC, GSA, GSV, VTG) provided position, time, altitude, speed, and satellite quality data.
- Statistical analysis of 5000 fixes yielded a 2D position spread of approximately 23.03 m (1-sigma). This value is larger than the manufacturer open-sky specification because the measurements were performed indoors near a window.

A Source Code Listings

A.1 ArduWaveshare.ino

```
1 #include <SoftwareSerial.h>
2
3 SoftwareSerial mySerial(10, 11); // RX=D10, TX=D11 (not connected)
4
5 void setup() {
6     Serial.begin(9600);
7     while (!Serial) { ; }
8     Serial.println("Hello! Setting up Software Serial.");
9     mySerial.begin(9600);
10    Serial.println("Hello, device?");
11 }
12
13 void loop() {
14     if (mySerial.available()) {
15         Serial.write(mySerial.read());
16     }
17 }
```

Listing 4: Complete Arduino firmware.

A.2 simpleNMEA.py

```
1 import serial
2
3 port      = '/dev/cu.usbserial-1470' # adjust to your system
4 baudrate = 9600
5 filename  = 'GNSS.txt'
6
7 try:
8     ser = serial.Serial(port, baudrate)
9 except:
10     print("something is wrong")
11
12 try:
13     with open(filename, "a") as file:
14         while True:
15             line = ser.readline().decode('ascii',
16                                     errors='replace').strip()
17             if line.startswith('$'):
18                 print(line)
19                 file.write(line + '\n')
20                 file.flush()
21 except KeyboardInterrupt:
22     print("\n\nStopped by user")
23 finally:
24     ser.close()
```

Listing 5: Complete NMEA logger.

A.3 NMEAReaderAccuracy.py

```
1 import serial
```

```

2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 port      = '/dev/cu.usbserial-1470'
6 baudrate  = 9600
7 max_fixes = 5000
8 longitude = []
9 latitude  = []
10
11 ser = serial.Serial(port, baudrate)
12 for n in range(max_fixes):
13     line = ser.readline().decode('ascii', errors='replace').strip()
14     if line.startswith('$'):
15         parts = line.split(',')
16         if parts[0].endswith('GGA') and len(parts) > 9:
17             degreeslat = int(parts[2][:2])
18             minuteslat = float(parts[2][2:])
19             degreeslong = int(parts[4][:3])
20             minuteslong = float(parts[4][3:])
21             decimallat = degreeslat + minuteslat / 60
22             decimallong = degreeslong + minuteslong / 60
23             latitude.append(decimallat)
24             longitude.append(decimallong)
25 ser.close()
26
27 lat_mean = np.mean(latitude)
28 lon_mean = np.mean(longitude)
29 print(f"Lat mean: {lat_mean:.6f}   std: {np.std(latitude):.2e}")
30 print(f"Lon mean: {lon_mean:.6f}   std: {np.std(longitude):.2e}")
31
32 lat_offset = (np.array(latitude) - lat_mean) * 1e5
33 lon_offset = (np.array(longitude) - lon_mean) * 1e5
34
35 fig, (ax1, ax2) = plt.subplots(2, figsize=(8, 6))
36 ax1.hist(lat_offset, bins='auto')
37 ax1.set_xlabel(r'Latitude deviation /  $\times 10^{-5}$  degrees')
38 ax1.set_ylabel('Number of measurements')
39 ax1.set_title('Latitude scatter')
40
41 ax2.hist(lon_offset, bins='auto')
42 ax2.set_xlabel(r'Longitude deviation /  $\times 10^{-5}$  degrees')
43 ax2.set_ylabel('Number of measurements')
44 ax2.set_title('Longitude scatter')
45
46 plt.tight_layout()
47 plt.savefig('gnss_accuracy_histograms.pdf', dpi=150)
48 plt.show()

```

Listing 6: Complete accuracy analysis script.

References

- [1] G. D. Greenwade, *The Comprehensive Tex Archive Network (CTAN)*, 1993.
- [2] R. Rettig, *Lecture 4: Bussystems and Sensors*, Lecture slides, 2026.